

Kibo Robot Programming Challenge - Team iTron Development Report

Ying-Pei Lin
NYCU, Hsinchu, Taiwan
kidalfonso@gmail.com

Dian-Mou Wu
NYCU, Hsinchu, Taiwan
drevild01801@gmail.com

Chang-Jyun Chen
NYCU, Hsinchu, Taiwan
jonathan0713tw@gmail.com

Li-Chi Cheng
NYCU, Hsinchu, Taiwan
Richby.cheng@gmail.com

July 2025

Abstract

This report presents the development of our system for the Kibo Robot Programming Challenge (Kibo RPC). Over the past few months, we designed the system architecture, implemented object detection, ran simulations, and analyzed performance. The report also includes our experience in both the preliminary and final rounds of the competition.

Our team achieved 1st place in the preliminary round, a result that validated the effectiveness of our strategy and implementation. Along the way, we faced many challenges, some of which seemed unsolvable at first. Through continuous research and teamwork, we managed to overcome them and build a functional, competitive system. This document serves not only as a technical record of our approach, but also as a reference manual for future developers who wish to understand or extend our modular design.

1 Introduction

1.1 Kibo-RPC Overview

The Kibo Robot Programming Challenge (Kibo-RPC) is a global robotics competition organized by the Japan Aerospace Exploration Agency (JAXA) and the National Aeronautics and Space Administration (NASA). It aims to promote interest and skills in space robotics by challenging students to write autonomous programs for Astrobees, a free flying robot operating inside the Japanese Experiment Module "Kibo" aboard the International Space Station (ISS).

The sixth Kibo-RPC is structured in two stages: a preliminary round held separately in each participating country or region, and a final round conducted aboard the ISS. In the preliminary round, teams develop and test their programs using JAXA's official web-based simulation environment. Each country selects its representative team. In the final round, the top teams from each region run their programs on the real Astrobees in orbit to determine the global winner.

Based on the official rules [1], each stage in the mission flow corresponds to the steps shown in Figure 1.

1. Astrobees start from the docking station.
2. After launching, it patrols four candidate locations aboard Kibo where hidden treasures might be found.
3. Each team selects a route through the Oasis Zones to earn additional points and reports the items detected at each candidate site.
4. Once all candidate locations have been visited, Astrobees approach the astronaut to scan an image of the real treasure and its nearby landmark, which reveals the true treasure's identity.
5. Astrobees then navigate to the real treasure and capture a photo.

6. Finally, the mission is completed by activating the Signal Lights to notify the astronaut of the treasure's location.

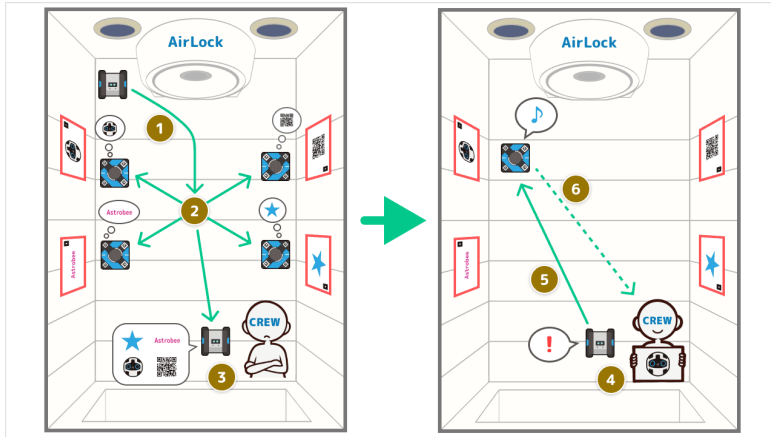


Figure 1: Mission flow of the 6th Kibo-RPC. Source: JAXA [1]

There are two types of items: landmarks and treasures. Images appear at random positions within each area, and each image will definitely contain several landmark items that must be reported by their type and quantity using the provided API. Treasures may or may not appear within these images. If a treasure appears, it should be remembered. After the astronaut identifies the real treasure, the robot needs to navigate to find that specific treasure. Figures 2 and 3 show the images of the landmark and treasure items, respectively.

Figure 2: Landmark item types. Source: JAXA [1]



(a) Coin



(b) Compass



(c) Coral



(d) Fossil



(e) Key



(f) Letter



(g) Shell



(h) Treasure Box

Figure 3: Treasure item types. Source: JAXA [1]



(a) Crystal



(b) Diamond



(c) Emerald

1.2 Team iTron

Our team, iTron, from the Department of Mechanical Engineering at National Yang Ming Chiao Tung University, is a student competition team passionate about mechanisms and control systems. In the 6th Kibo-RPC, we aimed to apply our technical expertise to a real-world space robotics scenario. With backgrounds in computer vision, motion planning, and embedded systems, we developed a modular architecture designed to perform reliably under limited and uncertain environments.

1.3 Report Overview

This report outlines the development of our Kibo-RPC system, covering team collaboration, toolchain choices, and key milestones. It details the system architecture, module implementations, integration process, and a review of our simulation results. Beyond the technical aspects, we aim to share insights into the problem-solving journey shaped by this international competition.

2 Development Process

2.1 Team Collaboration

We divided tasks based on each member’s expertise to ensure efficient development. The following Table 1 summarizes the responsibilities of each team member:

Table 1: Team Collaboration Responsibilities

Name	Responsibilities
Ying-Pei Lin	System Design, Training Data Preparation, Model Training, Preliminary Round Presentation
Dian-Mou Wu	System Design, Training Data Preparation, Performance Evaluation, Preliminary Round Presentation
Chang-Jyun Chen	Route Design, Training Data Preparation, Model Training
Li-Chi Cheng	Simulation, Performance Evaluation

2.2 Tools

We used Git for version control and hosted the project on GitHub. For early stage data preparation, we used label studio to organize training data. Later, we switched to data synthesis, automatically generating labels and splitting training and validation datasets, making the use of label studio unnecessary.

Although JAXA’s programming manual recommends using Android Studio for compilation, we set up the development environment using Docker and developed the project in Visual Studio Code (VSCode), which we are more familiar with.

We used Discord as our primary communication platform for team discussions, coordination, weekly meetings, and temporary file sharing.

2.3 Timeline

The development lasted around three months. In April and May, we focused on system design and integration. By the end of May, the system was mostly completed, and we continued to improve it by trying new strategy, updating the model, and searching better route. In June, before submitting the APK, we conducted extensive simulations to ensure the system was robust. The detailed project timeline is shown in Table 2.

Table 2: Project Timeline

Date / Period	Duration	Task
4/1	1 day	Jaxa releases the online simulator
4/1 – 4/7	1 week	Build the environment
4/8 – 4/28	3 weeks	Develop modules
4/29 – 5/13	2 weeks	Integrate and test modules
5/14 – 5/21	1 week	Implement actions after interacting with the astronaut
5/28 – 6/11	2 weeks	Test and update models based on simulation feedback
6/12 – 6/19	1 week	Finalize and submit APK
6/19	1 day	First Round APK Submission
7/13	1 day	Preliminary Round, Presentation

2.4 Milestones

During the development process, we iteratively improved our system through multiple releases. Each milestone introduced key updates to enhance detection accuracy, navigation stability, and overall performance. The following table summarizes our main release milestones:

Table 3: Release Milestones

Version	Date	Key Features
v1.0.0	2025-05-07	Initial release, score about 277 without level 4 pictures
v1.1.0	2025-05-10	New YOLO model trained with 2000 dataset, get 336 pt average in very hard mode
v1.2.0	2025-05-16	Combine area 2 and 3, update navigator pose for improved stability
v1.3.0	2025-05-23	Add WBF (Weighted Box Fusion), achieve 100% detection accuracy in very hard mode
v1.3.1	2025-05-27	Update pose to take area 1 image, improve score by 3 points on average
v1.3.2	2025-06-01	Refactor item detection thresholds and improve containment calculation
v1.3.3	2025-06-06	Update after tests, update flashlight brightness
v1.3.4	2025-06-08	Update new model and stable time
v1.3.5	2025-06-19	Adjust model m_61500 interpreter confidence threshold

After each new version release, we conducted 30 to 60 simulation runs to compute the average score and evaluate the model’s performance. The average scores over versions are shown in Figure 4.

There is a drop at version 1.2.0. This occurred because we changed the strategy to capture images of Area 2 and Area 3 simultaneously from a farther position than before. However, the model at that point was not good enough to handle the smaller and blurrier targets, resulting in lower detection accuracy and a decrease in the score. Following continuous improvements in the model, the scores began to increase again in later versions. In our latest version, 1.3.5, we achieved an average score of 286.8 points, with a detection precision of 99% across 30 simulation runs.

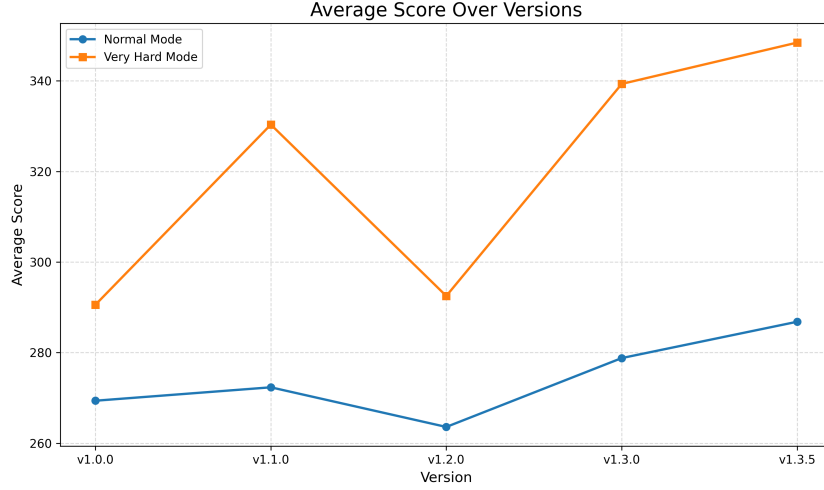


Figure 4: Average Score over Versions

3 System Design

3.1 Modules

Our program is organized into four layers. At the top layer, **YourService** serves as the default entry point of the framework, and **MainControl** executes the main task logic and allows modules interact with each other.

In the lower layer, **ItemManager** manages and stores detected items. **VisionHandler** acts as a wrapper for vision-related processes, images pass through a series of processing steps within VisionHandler to extract all necessary information. **Navigator** handles movement execution to control the robot's navigation.

In addition, we define two custom data structures, **Item** and **Pose**, to maintain a consistent data format and facilitate smooth data flow between modules.

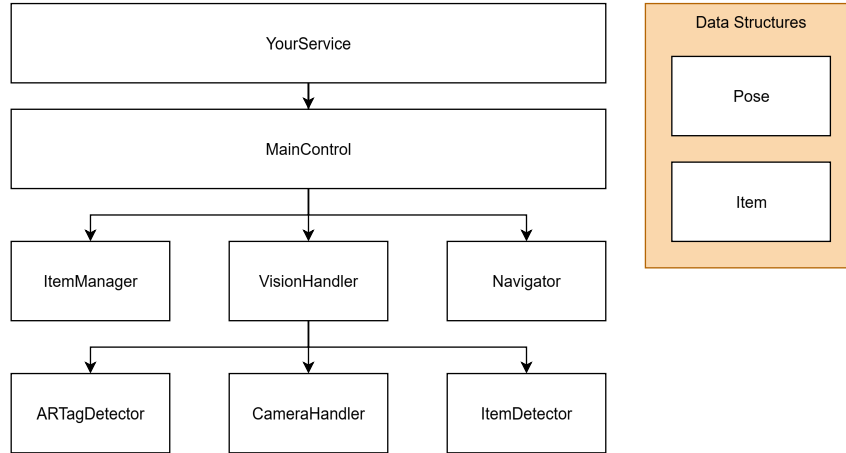


Figure 5: Code Structure

3.2 Task Flow

To complete the mission, our program guides Astrobbee through a sequence of image recognition and navigation tasks. The task flow is divided into four main stages:

1. **Initialization and mission start.** Astrobbee undocks from the docking station and initializes both its navigation and vision systems.

2. **Area patrol and item detection.** Astrobeer navigates to each of the four designated search areas to capture and analyze images using its front-facing NavCam. To optimize efficiency, we merge Area 2 and Area 3 into a single stop due to their closeness, allowing us to capture both with one photo. Each captured image is processed using our image pipeline, and the identified landmark and treasure items are saved and reported.
3. **Target item identification.** Astrobeer approaches the astronaut and captures the image held in their hand. The image is processed to identify the true treasure item, which is stored for later comparison.
4. **Treasure retrieval.** Using the name of the stored target item, Astrobeer compares it with the information recorded in each area. Once the correct area is determined, Astrobeer revisits that location and takes a final snapshot of the treasure to complete the mission.

3.3 Image Pipeline

Our image pipeline includes several stages, as shown in Figure 6.

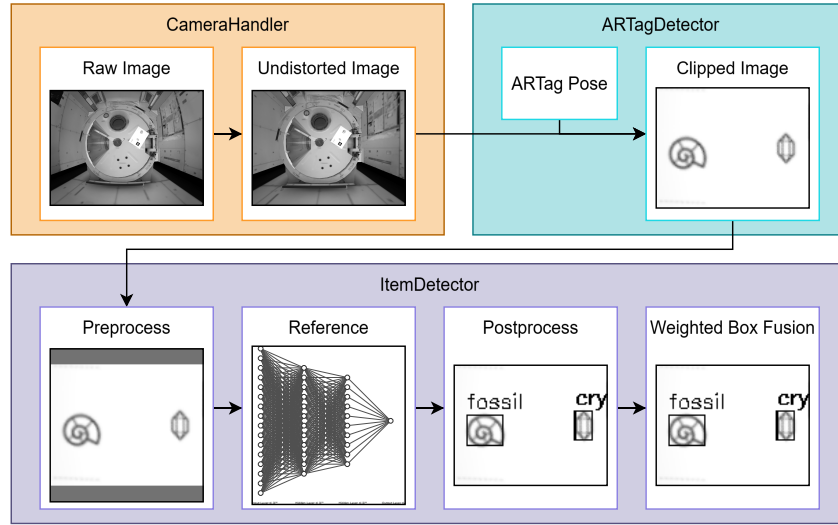


Figure 6: Image Pipeline

First, the *CameraHandler* captures a raw image and undistorts it using camera calibration parameters. The raw image and the undistorted image are quite similar, and the differences are near the corners. The second one is calibrated, so the line near the corner remains straight, whereas these lines appear curved due to lens distortion in the raw image.

Next, the *ARTagDetector* extracts the AR tag information, it contains the relative coordinates of the tag in the OpenCV camera frame. We transform the relative coordinates into absolute world coordinates based on Astrobeer’s pose in world frame. The output pose from Algorithm 1 is later stored together with the detected items in *ItemManager*. The calculations are shown below.

Algorithm 1 Transform Tag Pose from OpenCV Camera Frame to World Frame

Require: P_{tag}^{cv} : Pose of tag in OpenCV camera frame

Require: T_{body}^{world} : Transformation of body frame to world frame

Ensure: P_{tag}^{world} : Pose of tag in world frame

- 1: Define transformation of OpenCV camera frame to NavCam frame, T_{cv}^{nav}
 - 2: Define transformation of NavCam frame to body frame, T_{nav}^{body}
 - 3: $P_{tag}^{nav} \leftarrow T_{cv}^{nav} \cdot P_{tag}^{cv}$
 - 4: $P_{tag}^{body} \leftarrow T_{nav}^{body} \cdot P_{tag}^{nav}$
 - 5: $P_{tag}^{world} \leftarrow T_{body}^{world} \cdot P_{tag}^{body}$
 - 6: **return** P_{tag}^{world}
-

We also use the relative coordinates to crop the image to focus on the region of interest with Algorithm 2 before it passed to the *ItemDetector*.

Algorithm 2 Clip Image to Area of Interest

Require: Image I , Corner points matrix C (4 corner points)

Ensure: Clipped image I_{clip}

```

1: Initialize array points of length 4
2: for  $i = 0$  to 3 do
3:    $\text{points}[i] \leftarrow$  coordinates from  $C$  at index  $i$ 
4: end for
5: Extract corners as:
6:  $c_0 \leftarrow C[:, 0], \quad c_1 \leftarrow C[:, 1], \quad c_2 \leftarrow C[:, 2], \quad c_3 \leftarrow C[:, 3]$ 
7: Set transformation parameters:
8:  $h_0 \leftarrow -4.4, \quad h_1 \leftarrow 0.0, \quad v_0 \leftarrow -0.5, \quad v_1 \leftarrow 3.0$ 
9: Compute transformed points:
10:

$$\begin{aligned} \text{points}[0] &\leftarrow h_0(c_1 - c_0) + v_0(c_3 - c_0) + c_0 \\ \text{points}[1] &\leftarrow h_1(c_1 - c_0) + v_0(c_3 - c_0) + c_0 \\ \text{points}[2] &\leftarrow h_1(c_1 - c_0) + v_1(c_3 - c_0) + c_0 \\ \text{points}[3] &\leftarrow h_0(c_1 - c_0) + v_1(c_3 - c_0) + c_0 \end{aligned}$$

11: Calculate width and height:
12:  $\text{width} \leftarrow \sqrt{(\text{points}[0].x - \text{points}[1].x)^2 + (\text{points}[0].y - \text{points}[1].y)^2}$ 
13:  $\text{height} \leftarrow \sqrt{(\text{points}[0].x - \text{points}[3].x)^2 + (\text{points}[0].y - \text{points}[3].y)^2}$ 
14: Define source points matrix  $S \leftarrow \text{points}$ 
15: Define destination points matrix  $D \leftarrow \{(0, 0), (\text{width} - 1, 0), (\text{width} - 1, \text{height} - 1), (0, \text{height} - 1)\}$ 
16: Compute perspective transform matrix  $T \leftarrow \text{getPerspectiveTransform}(S, D)$ 
17: Apply warp perspective:
18:  $I_{clip} \leftarrow \text{warpPerspective}(I, T, (\text{width}, \text{height}))$ 
return  $I_{clip}$ 

```

In *ItemDetector*, the cropped image undergoes preprocessing. It is first converted from **Mat** to **Bitmap**, then resized to fit the model’s input size using letterbox padding to maintain aspect ratio. The resized image is loaded into a **TensorImage**, where it is normalized and converted to the correct data type. Finally, the processed image is extracted as a **ByteBuffer** and fed into the YOLO model for inference.

In postprocessing, the model’s output tensors are reorganized to reconstruct bounding boxes and filtered by confidence scores. Only detections above a predefined threshold are kept, resulting in a list of items with their bounding boxes and confidence scores for further processing.

The detections are then refined using a revised weighted box fusion method to improve accuracy, as detailed in Algorithm 3. In the algorithm, right before merging start, we sort detections by confidence in descending order so that high confidence results are prioritized in the merging process.

Unlike traditional WBF, which only merges boxes of the same class, our revised method merges all nearby boxes, regardless of class, and assigns the final class based on the highest confidence among those merged. This approach compensates for early stage models with low precision. The idea is that even if the class prediction is wrong, the location can still be accurate and useful. By ensembling multiple models, we improve robustness and let confidence guide the final decision.

However, as our models became more accurate in later stages, the benefit of using our revised WBF became smaller. The results were almost the same as using the standard WBF or even simple Non-Maximum Suppression (NMS).

Algorithm 3 Revised Weighted Box Fusion (WBF)

Require: D : List of detection results with class, confidence, box and model weights
Require: T_{IoU} : IoU threshold for merging boxes
Require: T_{cont} : Containment threshold for merging boxes
Require: T_{conf} : Confidence threshold for filtering boxes
Ensure: Fused list of detection boxes F

```
1: if  $D$  is empty then
2:   return  $D$ 
3: end if
4: Sort  $D$  by confidence in descending order
5: Initialize  $F \leftarrow \emptyset$ ,  $U \leftarrow$  array of false (used flags)
6: for each  $d_i$  in  $D$  do
7:   if  $U[i]$  is true then
8:     continue
9:   end if
10:  Initialize group  $G \leftarrow \{d_i\}$ , set  $U[i] \leftarrow$  true
11:  for each  $d_j$  in  $D$  where  $j > i$  and  $U[j]$  is false do
12:    if  $\text{IoU}(d_i.\text{box}, d_j.\text{box}) > T_{IoU}$  or  $\text{Containment}(d_j.\text{box}, d_i.\text{box}) > T_{cont}$  then
13:      Add  $d_j$  to  $G$ , set  $U[j] \leftarrow$  true
14:    end if
15:  end for
16:  Initialize  $\text{sum\_conf} \leftarrow 0$ ,  $\text{sum\_weight} \leftarrow 0$ 
17:  Initialize  $\text{fused\_box} \leftarrow (0, 0, 0, 0)$ 
18:  for each  $d_j$  in  $G$  do
19:     $\text{sum\_conf} \leftarrow \text{sum\_conf} + d_j.\text{conf} \times d_j.\text{weight}$ 
20:     $\text{sum\_weight} \leftarrow \text{sum\_weight} + d_j.\text{weight}$ 
21:     $\text{fused\_box} \leftarrow \text{fused\_box} + d_j.\text{box} \times d_j.\text{conf} \times d_j.\text{weight}$ 
22:  end for
23:  Compute  $\text{avg\_conf} \leftarrow \text{sum\_conf} / \text{sum\_weight}$ 
24:  if  $\text{avg\_conf} < T_{conf}$  then
25:    continue
26:  end if
27:  Let  $(x1, y1, x2, y2) \leftarrow \text{fused\_box}$ 
28:  Normalize  $x1, y1, x2, y2$  by dividing each by  $\text{sum\_conf}$ 
29:  Compute  $x\_min \leftarrow \min(x1, x2)$ ,  $x\_max \leftarrow \max(x1, x2)$ 
30:  Compute  $y\_min \leftarrow \min(y1, y2)$ ,  $y\_max \leftarrow \max(y1, y2)$ 
31:  Compute  $\text{box\_width} \leftarrow x\_max - x\_min$ ,  $\text{box\_height} \leftarrow y\_max - y\_min$ 
32:  if  $\text{box\_width}$  or  $\text{box\_height} < 5\%$  then
33:    continue
34:  end if
35:  Sort  $G$  by weighted confidence in descending order
36:  Let  $d_{best} \leftarrow G[0]$ 
37:  Add merged detection  $(d_{best}.\text{class}, \text{avg\_conf}, (x\_min, y\_min, x\_max, y\_max))$  to  $F$ 
38: end for
39: return  $F$ 
```

The final results are stored in *ItemManager*.

3.4 Navigation Strategy

Although the final bonus score depends mostly on the total task time, which is calculated by the function

$$\text{time_bonus_score} = \text{total_rest_time} * 0.2$$

while the formula for the oasis bonus score remains unknown. What we do know is that the farther away from the area, the higher the score becomes. The Figure 7 illustrates what the potential oasis score distribution might look like.

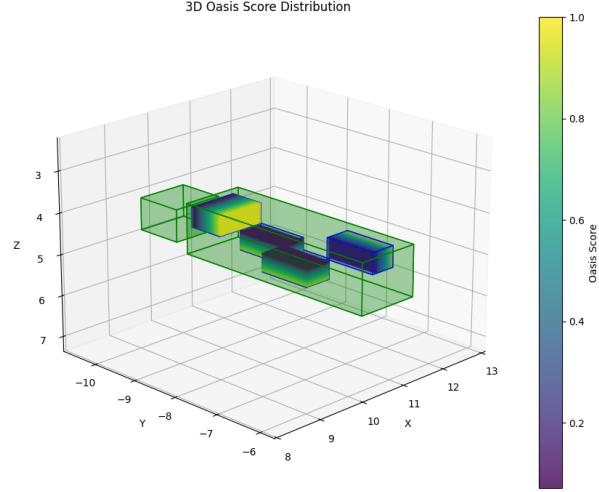


Figure 7: 3D Oasis Score Distribution

We observed that the oasis bonus typically ranges from 0.5 to 15 points, depending on how far the robot detours to pass through the oasis zone. Initially, we focused on minimizing task time, which generally produced a higher score than attempting to collect oasis bonuses. However, once our score reached around 280 (in normal mode), further optimizing task time became difficult, gaining just 1 additional point required reducing the task time by 5 seconds, which was challenging.

Through testing, we discovered that sacrificing a few seconds (e.g. 10 seconds, equivalent to 2 points lost from the time bonus) to make a strategic detour could earn more than 5 points in oasis bonus. This insight shifted our strategy toward balancing time efficiency and oasis detours.

Our final navigation route reflects this optimized trade-off, as shown in Figure 8.

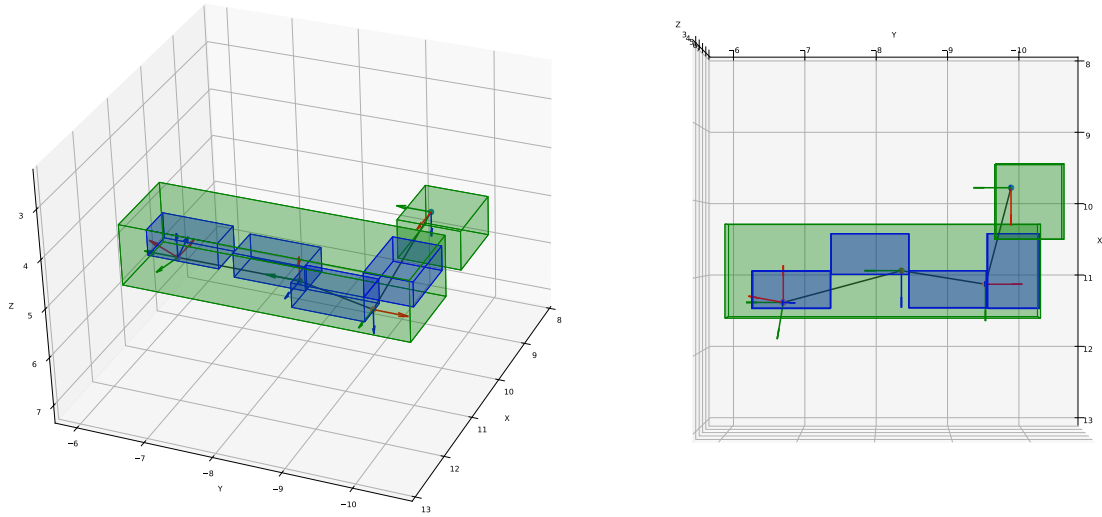


Figure 8: Route Design

We first captured the image of Area 1 from the bottom of Oasis 2. Then, we moved to the bottom of Oasis 3 and captured Areas 2 and 3 simultaneously from that position. After that, we moved to Oasis 4 to take the image of Area 4. Finally, we only changed Astrobees quaternion to face the astronaut, which is why you can see two overlapping local axes at Oasis 4. The positions and orientations are shown in Table 4.

Table 4: Positions and Orientations of Areas

Area	Position (x, y, z) [m]	Quaternion (x, y, z, w)
1	(11.11, -9.49, 5.435)	(0.0, 0.0, -0.707, 0.707)
2 & 3 combined	(10.925, -8.35, 5.3)	(0.0, 0.707, 0.0, 0.707)
4	(11.3, -6.76, 4.935)	(0.0, -1.0, 0.0, 0.0)
0 (Report)	(11.3, -6.76, 4.935)	(0.633, 0.754, -0.133, 0.112)

4 Object Detection

4.1 Training Data Preparation

There are 10 different objects that need to be recognized during the competition. Since we developed Algorithm 2 to directly extract the regions where objects appear, our dataset was designed using a white background, with objects of random sizes placed at random positions. The dataset generation was divided into two key parts.

First, in the general case, objects are normally distributed with minimal overlap. The goal here was to train the model to distinguish between the 10 different object types.

Second, each image contains only a single object, with more extreme variations in size and position compared to the general case. The purpose of this setup was to help the model learn the distinctive features of each object type, thereby improving its ability to recognize them even when multiple objects are heavily overlapped.

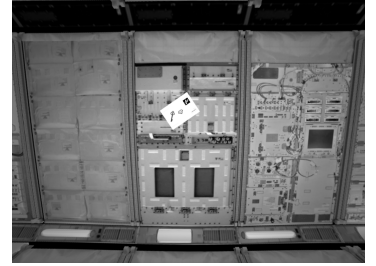
To improve the model’s robustness and generalization, we applied common image augmentation techniques. These included random brightness changes to simulate different lighting, and adding Gaussian blur and noise to mimic imperfections in image capture. These augmentations help the model stay effective even when the input images are not ideal.



(a) Synthetic Dataset

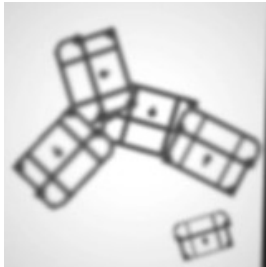


(b) Clipped Image



(c) Camera

Figure 9: General Case



(a) Synthetic Dataset



(b) Clipped Image



(c) Camera

Figure 10: Overlapping Case

4.2 Model Training

After generating the dataset, we performed compression and dataset splitting. The images—totaling 61,500—were compressed from PNG to WebP format to significantly reduce file size and improve I/O

performance during training. The dataset was then split into training, testing, and validation sets in a 7:2:1 ratio.

For model selection and training, we used Ultralytics’ YOLOv11 (You Only Look Once version 11), a state-of-the-art object detection architecture optimized for both accuracy and speed. YOLOv11’s ability to handle small objects and dense scenes made it particularly well-suited for our task. We configured the training parameters with 180 epochs and an image size of 160×160 , balancing the need to capture small object features with the computational constraints of large-scale training.

To enhance the generalization ability of the model, we incorporated extensive data augmentation, including random brightness adjustments, Gaussian blur, and noise injection. Additionally, we applied random rotations, horizontal flips, and slight scale jittering to simulate real-world variability and improve robustness.

Training was conducted on high-performance GPUs provided by KTH as well as personal GPUs owned by team members. Given the large dataset size, we ensured proper shuffling and class balance in each batch to prevent bias and promote stable convergence.

Once the trained model met our performance requirements, evaluated based on precision, recall, and mAP metrics, we proceeded with the deployment phase. The PyTorch model was first exported to ONNX format, and then converted into TensorFlow Lite using the appropriate optimization flags for quantization and inference speed. This allowed efficient integration with the competition simulator while maintaining real-time performance.

5 Competition Participation

5.1 Preliminary Round

Our team achieved 1st place in the Taiwan regional preliminary round, earning the opportunity to participate in the final round aboard the International Space Station.



(a) Receiving the Preliminary Round award 1



(b) Receiving the Preliminary Round award 2

Figure 11: Scenes from the Preliminary Round
Image source: Kibo-RPC Taiwan Organizer [2].

6 Future Work

Although the system is generally reliable, some minor issues remain that rarely affect performance. One example is when items appear at the very edge of the image. The rule requires that the item’s final alignment must be within 30 degrees of the robot’s forward view. However, if the item is too close to the image boundary, our current method, which aligning to the tag, might cause the item itself to be slightly misaligned, leading the simulator to mark it as a failure.

In most cases, the item is not far from its tag, so this approach works fine. A more accurate method would involve adjusting the tag’s pose toward the image center or estimating the item’s exact pose through additional calculations. We discovered this issue in late May, but since it only occurs about once in a hundred runs, we decided not to address it during the competition.

7 Conclusion

In this report, we have detailed the design, development, and testing of our system for the 6th Kibo Robot Programming Challenge. Our team, iTron successfully implemented a modular architecture combining computer vision, navigation strategies, and autonomous decision making under constrained conditions.

Throughout the three month development process, we overcame challenges related to object detection accuracy, image distortion correction, and optimal navigation planning. We proposed a revised weighted box fusion algorithm to further enhance detection robustness and consistency. Our final system achieved an average simulation score of 286.8 points (the highest in our region) with 99% detection accuracy across multiple test runs.

Beyond the technical achievements, participating in the Kibo-RPC provided invaluable experience in problem solving, team collaboration, and applying theoretical knowledge to real-world space robotics scenarios. We hope the documentation of our journey can serve as a useful reference for future participants and teams aiming to tackle similar challenges in autonomous robotics development.

8 Acknowledgements

We would like to express our sincere gratitude to KTH Cloud for providing high-end remote computing resources during our team member Ying-Pei Lin's exchange program in Sweden. With their support, we had the opportunity to turn our ideas into reality.

Division of Work

Name	Responsibilities	Signature
Ying-Pei Lin	System Design, Training Data Preparation, Model Training, Preliminary Round Presentation	林穎沛
Dian-Mou Wu	System Design, Training Data Preparation, Performance Evaluation, Preliminary Round Presentation	吳典謀
Chang-Jyun Chen	Route Design, Training Data Preparation, Model Training	陳昌駿
Li-Chi Cheng	Simulation, Performance Evaluation	鄭力齊

Supervisor Signature

Name: Teng-Hu Cheng

Signature: 

Date: 2025/08/08

References

- [1] JAXA. Kibo robot programming challenge. <https://jaxa.krpc.jp/>. Accessed: 2025-08-05.
- [2] Kibo-RPC Taiwan Organizer. Preliminary round award ceremony photos, 2025. Shared via private Google Drive link. Accessed: July 2025.